斜率优化/凸壳优化

有一类DP状态方程:

$$dp[i] = min\{dp[j] - a[i] * d[j]\}, 0 \le j < i, d[j] \le d[j+1], a[i] \le a[i+1]$$

- 特征:存在一个既有i又有j的项$a[i] * d[j]$。
- 编程时，如果简单地对外层i和内层j循环，复杂度是$O(n^2)$。这里能用单调队列优化吗？单调队列所处理的策略，要求只能与内层j有关，与外层i无关，但是这个状态方程无法简单地得到只与j有关的部分。
- 用斜率(凸壳)模型，能够把方程转化，得到一个只和j有关的部分，即“斜率”，从而能够使用单调队列优化。

1. 把状态转移方程变换为平面的斜率问题

观察状态转移方程，可以把i看作不变的常量，把除了dp[i]以外的关于i的部分看作常量，把j看作变量，目标是求j变化时dp[i]的最优值。dp[i]的变化对应了变化的直线截距。

去掉min，把状态转移方程转换为

$$dp[j] = a[i] * d[j] + dp[i]$$

为了方便观察，令 $y = dp[j]$， $x = d[j]$， $k = a[i]$， $b = dp[i]$ ， 状态转移方程变为$y = kx + b$

斜率优化的数学模型，就是把状态转移方程转换为平面坐标系直线的形式$y = kx + b$。

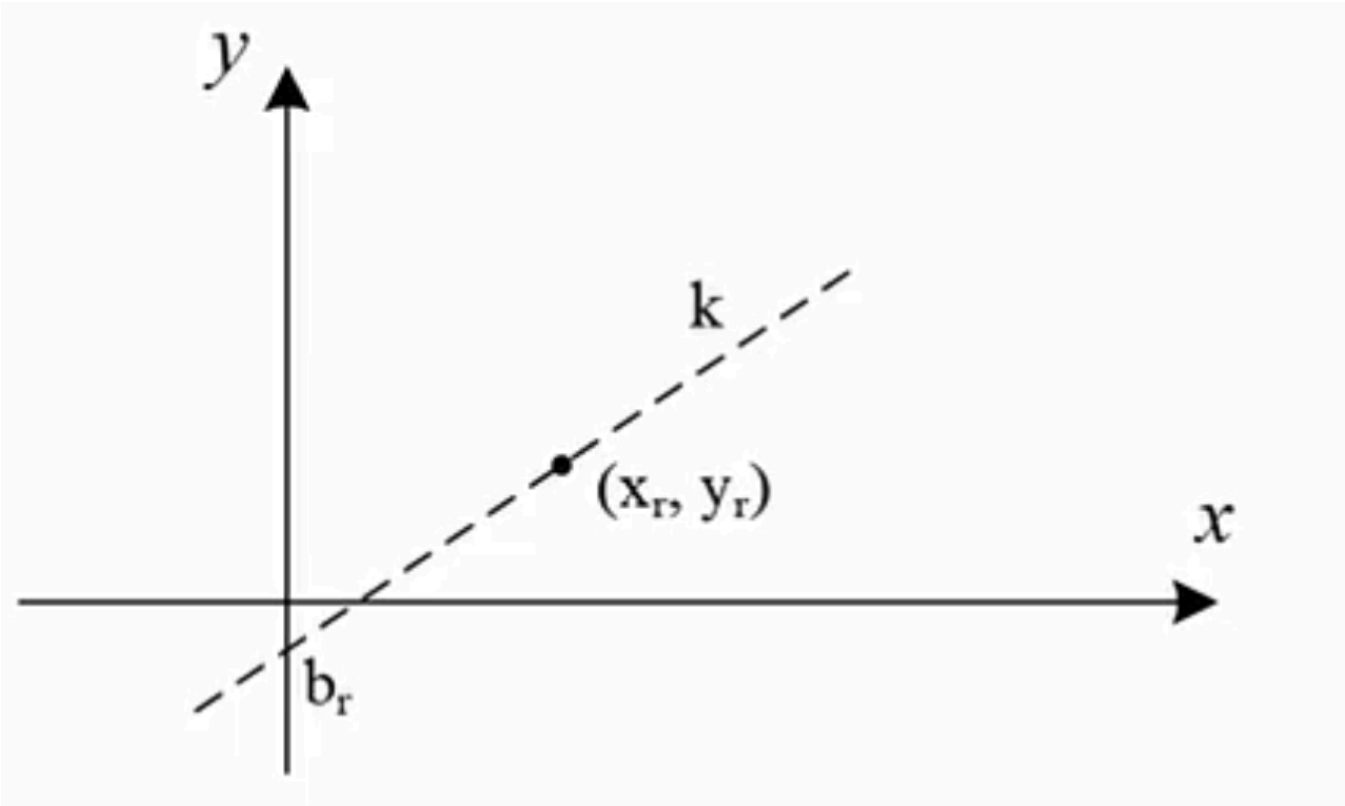
- (1)变量x、y与j有关，并且只有y中包含dp[j]。点(x,y)是题目中可能的决策。
- (2)斜率k、截距b与i有关，并且只有b中包含dp[i]。最小的b包含最小的dp[i]，也就是状态方程的解。

两点(x_1, y_1)、(x_2, y_2)连成的直线的斜率为$(y_2 - y_1)/(x_2 - x_1)$，它们只与j有关，这就是可以用单调队列处理的策略。

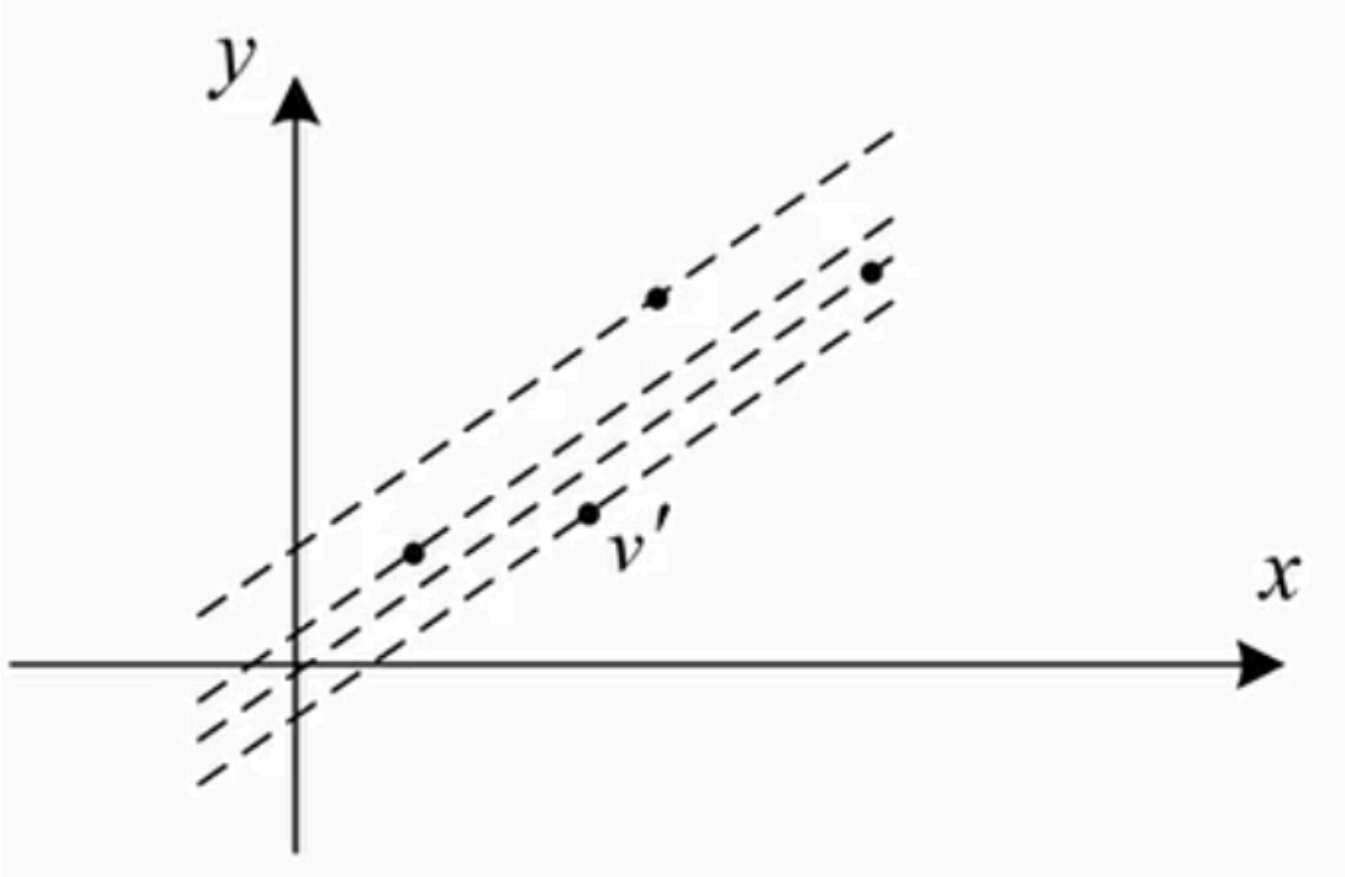
应用斜率优化的条件为x和k是单调增加的，即x随着j递增而递增，k随着i递增而递增。

2. 求一个dp[i]

固定i的情况下求dp[i]。由于i是定值，斜率$k = a[i]$可以看成常数。当j在 $0 \le j < i$ 内变化时，对某个j_r ，产生一个点v_r=(x_r，y_r)，这个点在一条直线$y = kx + b$上，b_r是截距。

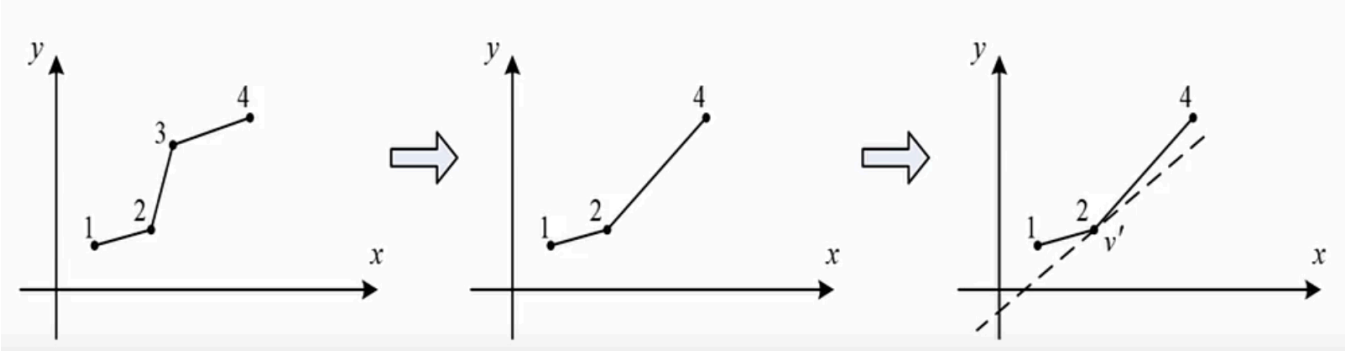


对于 $0 \le j < i$ 中所有的 j，把它们对应的点都画在平面上，这些点对应的直线的斜率$k = a[i]$都相同，只有截距b不同。



在所有这些点中，有一个点v'所在的直线有最小截距b'，算出b'，由于b'中包含dp[i]，那么就算出了最优的dp[i]。

如何找最优点v'?利用“下凸壳”。



x是单调增加的，即x随着j递增而递增。

左图中的1、2、3构成了“下凸壳”，“下凸壳”的特征是线段12的斜率小于线段23的斜率。2、3、4构成了“上凸壳”。经过上凸壳中间点3的直线，其截距b肯定小于经过2或4的有相同斜率的直线的截距，所以点3肯定不是最优点，去掉它。

去掉“上凸壳”后，得到中间的图像，留下的点都满足“下凸壳”关系。最优点就在“下凸壳”上。例如在右图中，用斜率为k的直线来切这些点，设线段12的斜率小于k，24的斜率大于k，点2就是“下凸壳”的最优点。以上操作用单调队列编程很方便。

3. 求所有的dp[i]

- 一个dp[i]，复杂度O(n)。如果对所有的i，每一个都这样求dp[i]，总复杂度仍然是$O(n^2)$的。有优化的方法吗?
- 一个较小的i_1，它对应的点是{v_0， v_1， ...， v_{i1}}。
- 一个较大的i_2，对应了更多的点{v_0， v_1， ...， v_{i1}， ...， v_{i2}}，其中包含了i_1的所有点。
- 寻找i_1的最优点时，需要检查{v_0， v_1， ...， v_{i1}}
- 寻找i_2的最优点时，需要检查{v_0， v_1， ...， v_{i1}， ...， v_{i2}}
- 这里做了重复的检查，并且这些重复是可以避免的。这就是能优化的地方，仍然用“下凸壳”进行优化。

4. 例题 Print article（HDU 3507）

题意

打印一篇包含 N 个单词的文章，第i个单词的打印成本为C_i。在一行中打印k个单词的花费为$\left(\sum_{i=1}^{k} C_i\right)^2 + M$， M为一个常数。如何安排文章，才能最小化费用?

输入

有很多测试用例。对于每个测试用例，第1行输入两个数字N和M, $(0 \le N \le 500000, 0 \le M \le 1000)$。在接下来的第$2 \sim N+1$行中，输入$N$个数字。输入$EOF$终止。

输出

打印文章的最低费用。

输入样例

```
5 5
5
9
5
7
5
```

输出样例

```
230
```

思路

题意是有N个数和一个常数M，把这N个数分成若干部分，每部分的计算值为这部分数的和的平方加上M，总计算值为各部分计算值之和，求最小的总计算值。由于N很大，$O(N^2)$的算法超时。

设$dp[i]$表示输出前i个单词的最小费用，DP状态转移方程为

$$dp[i] = min\{dp[j] + (sum[i] - sum[j])^2 + M\}, 0 < j < i$$

其中，$sum[i]$表示前i个数字和。

刚看到这个转移方程可能不容易想出来如何用斜率优化，**同学们在阅读下面内容之前可以先自己尝试如何套用$y = kx + b$这种斜率优化DP的标准形式。**

下面讲DP状态转移方程改写为$y = kx + b$这种斜率优化DP的标准形式。首先展开方程，得

$$dp[i] = dp[j] + sum[i]sum[i] + sum[j]sum[j] - 2sum[i]sum[j] + M$$

移项得到

$$dp[j] + sum[j]sum[j] = 2sum[i]sum[j] + dp[i] - sum[i]sum[i] - M$$

对照$y = kx + b$，有

- $y = dp[j] + sum[j]sum[j]$，y只与j有关；
- $x = 2sum[j]$，x只与j有关，且随着j递增而递增；
- $k = sum[i]$，k只与i有关，且随着i递增而递增；
- $dp[i] - sum[i]sum[i] - M$，b只与i有关，且包含$dp[i]$。

下面给出代码。注意一个小技巧：虽然在理论上单调队列处理的是斜率$(y(a) - y(b))/(x(a) - x(b))$，但是编码时不用这样算，队列只记录$a$和$b$，判断时用$(y(a) - y(b))/(x(a) - x(b))$计算即可。细节详见代码。

参考代码

```
#include <iostream>
using namespace std;
const int N = 500010;
int dp[N];
int q[N];      //单调队列
int sum[N];
int X(int x){ return 2*sum[x]; }
int Y(int x){ return dp[x]+sum[x]*sum[x]; }
//double slope(int a,int b){return (Y(a)-Y(b))/(X(a)-X(b));}//除法不好，改成下面的乘法
int slope_up  (int a,int b) { return Y(a)-Y(b);}   //斜率的分子部分
int slope_down(int a,int b) { return X(a)-X(b);}   //斜率的分母部分
int main(){
    int n,m;
    while(~scanf("%d%d",&n,&m)){
        for(int i=1;i<=n;i++)  scanf("%d",&sum[i]);
        sum[0] = dp[0] = 0;
        for(int i=1;i<=n;i++)  sum[i]+=sum[i-1];
        int head=1,tail=1;      //队头队尾
        q[tail]=0;
        for(int i=1;i<=n;i++){
            while(head<tail &&
                  slope_up(q[head+1],q[head])<=sum[i]*slope_down(q[head+1],q[head]))
               head++;           //斜率小于k，从队头弹走
            int j = q[head];     //队头是最优点
            dp[i] = dp[j]+m+(sum[i]-sum[j])*(sum[i]-sum[j]);    //计算dp[i]
            while(head<tail &&
                slope_up(i,q[tail])*slope_down(q[tail],q[tail-1])
                <= slope_up(q[tail],q[tail-1])*slope_down(i,q[tail]))
                tail--;          //弹走队尾不合格的点
            q[++tail] = i;       //新的点进队尾
        }
        printf("%d\n",dp[n]);
    }
    return 0;
}
```